Elevator Control System – Design, Verification, and Validation Report

Name : Ruchika Verma

Group 5

Course: 1DT059 – Model-based Design of Embedded Software

---

1. Introduction

The objective of this project was to design, implement, and verify an elevator control system using **MATLAB Simulink and State-Flow**, with support for both **pure simulation** and **hardware execution using Arduino**. The elevator serves four floors (floors 1 to 4) and supports basic user interaction such as floor requests, door operation, request cancellation, and emergency stop.

The focus of the project was not on building a mechanically accurate elevator, but on developing a **correct, verifiable, and understandable controller** that satisfies a set of formally defined requirements. Formal verification techniques and automated test generation were used to check correctness.

In the final stage of the project, the elevator controller was also tested using real hardware. An ultrasonic distance sensor connected to the Arduino was used to estimate the elevator position during motion. The measured distance was mapped to floor positions and used by the controller to decide when to stop and open the door. This allowed the behaviour of the controller to be validated not only in simulation, but also in a real execution setup.

---

2. Overall Design Effort and Model Organization

Instead of building a single large Simulink model, the system was divided into multiple models, each serving a specific purpose. This decision was made to avoid tool limitations, reduce complexity, and clearly separate design, simulation, hardware execution, and verification.

The following models were developed:

1. **Event-driven State-Flow model**
   Used in the early stage to design and validate controller logic using State-Flow events and dashboard buttons.

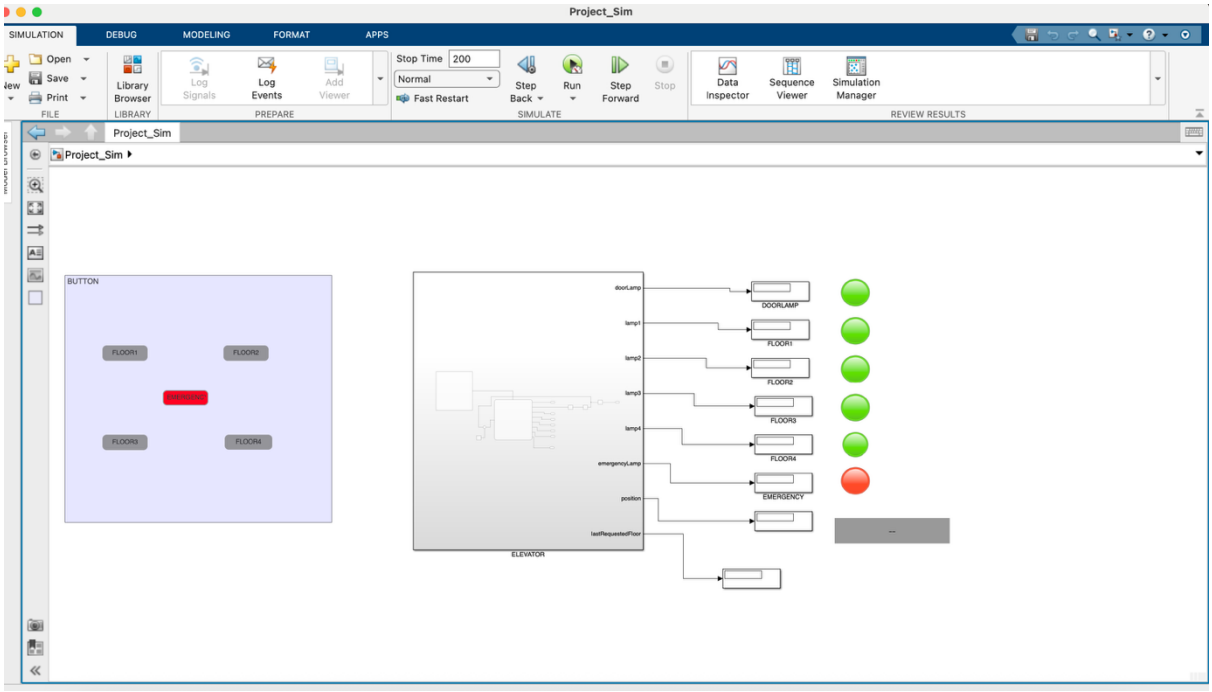


Figure 1:EventDriven_Top_Level_Model

2. **Pure Simulink simulation model**
Contains the controller and a simplified elevator physics model. Used for simulation, logging, and replay of input signals.

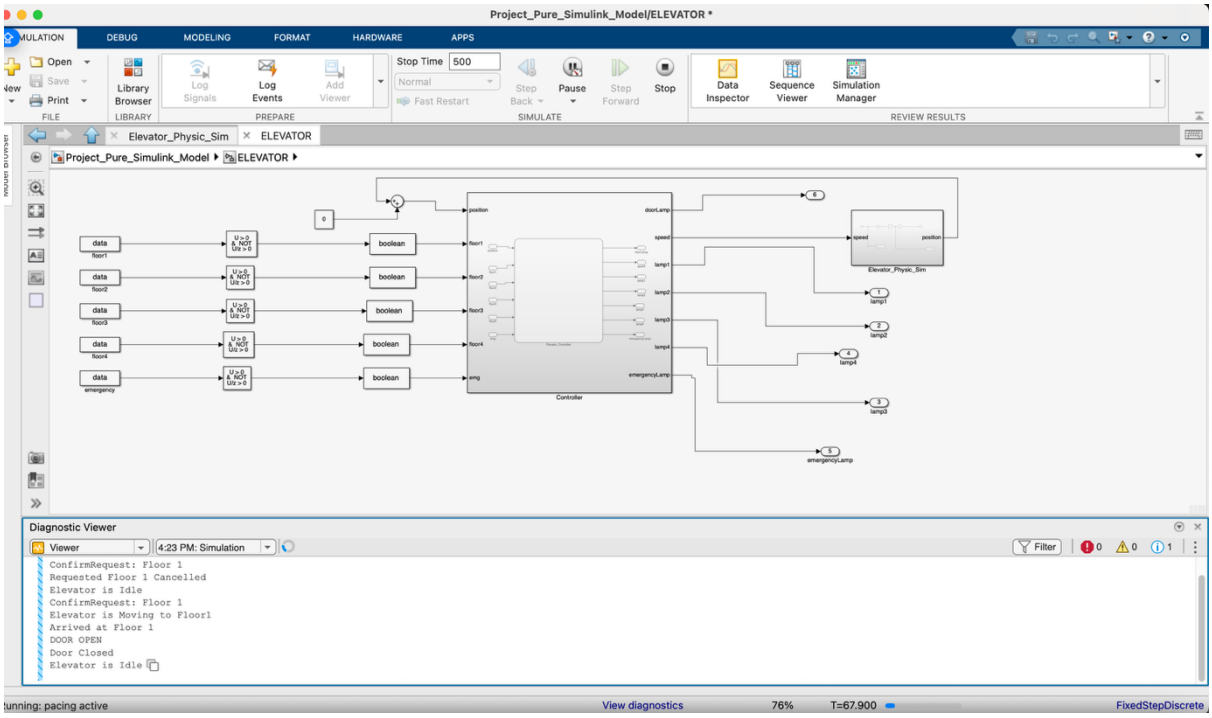


Figure 2:Simulink_Top_Level_Model

3. **Arduino hardware model**
Integrates the controller with Arduino digital input/output blocks and motor driver signals.

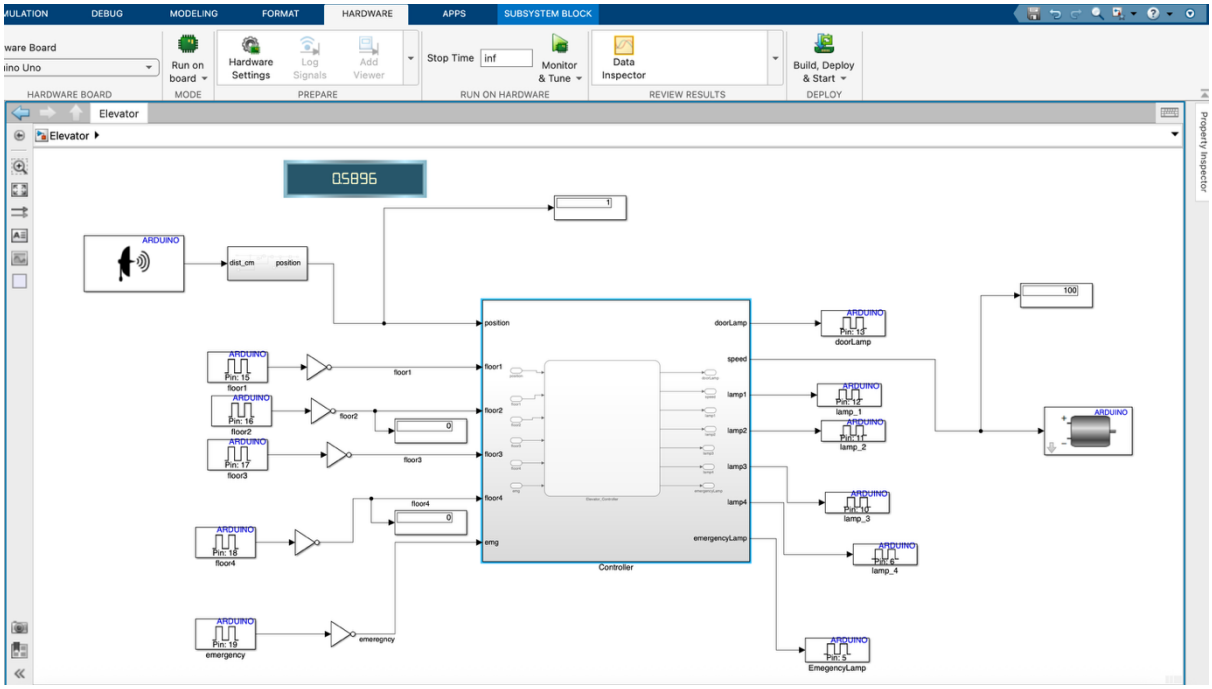


Figure 3:Arduino_Top_Level_Model

4. **Requirement monitoring model**
   A separate model containing State-Flow monitors for formal requirements R1–R4.

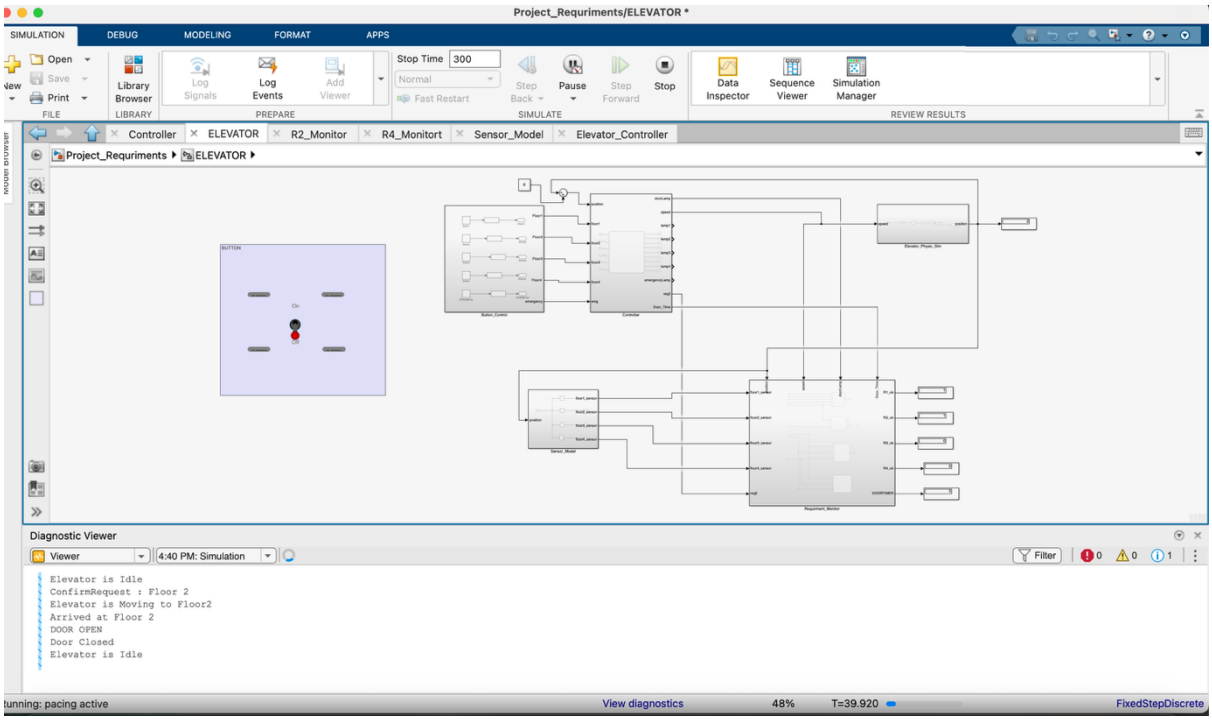


Figure 4:Requriment_Top_Level_Model

5. **SLDV verification model**
   A simplified, fully discrete model compatible with Simulink Design Verifier, used for automated test generation and coverage analysis.

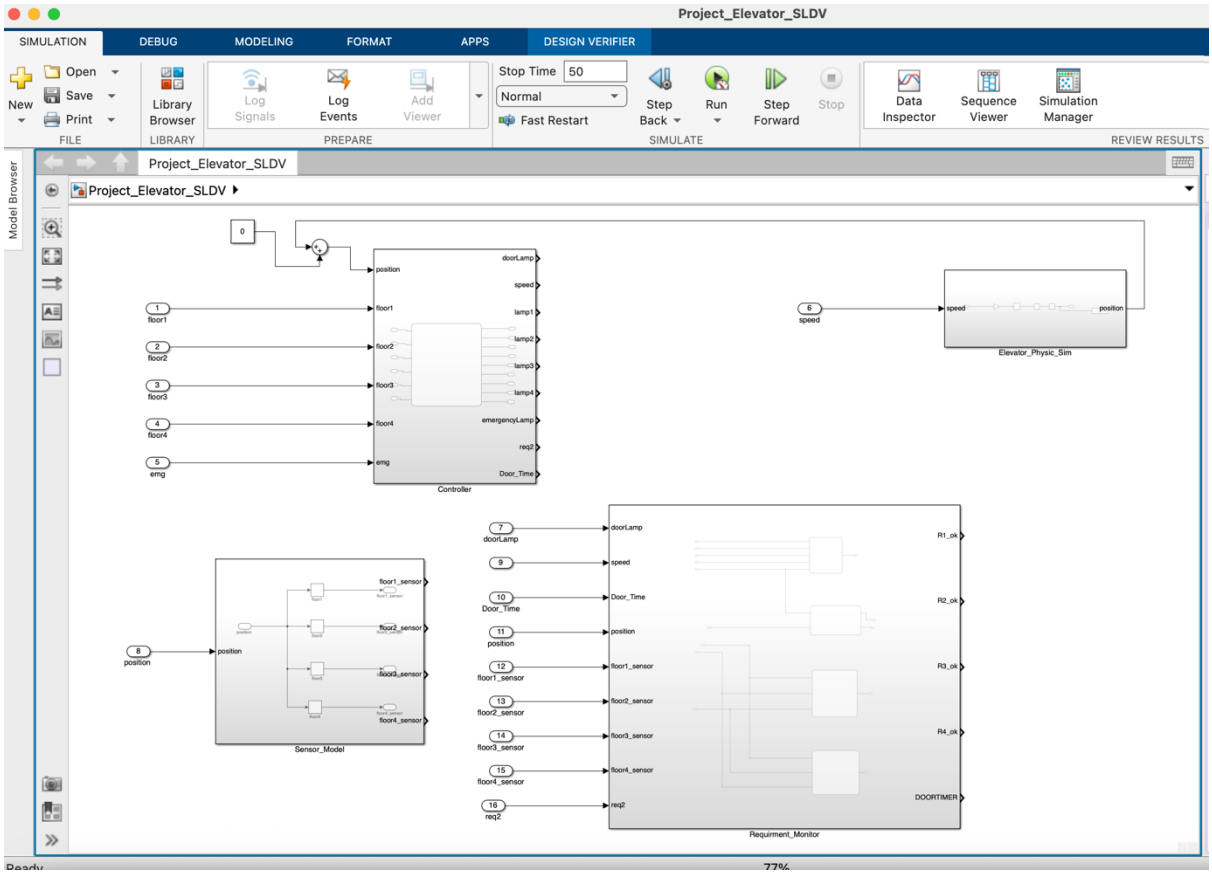


Figure 5:SLDV_Top_Level_Model

This modular structure made debugging, verification, and explanation of the system easier and avoided incompatibilities between different toolchains.

---

3. Controller Design

3.1. Controller Structure

The elevator controller was implemented using **State-Flow**, as elevator behaviour is naturally state-based. The controller consists of five main states:

- **Idle**
  The elevator is stationary and waits for a user request.
- **ConfirmRequest**
  After a button press, the system waits for a short time window to allow cancellation of the request.
- **Moving**
  The elevator moves upward or downward depending on the requested floor.
- **DoorOpen**
  The elevator stops at the target floor and opens the door for a fixed duration.

- **Emergency**
  The elevator stops immediately and remains stopped until the emergency condition is cleared.

Transitions between these states are driven by button inputs, timing conditions, position checks, and emergency signals.

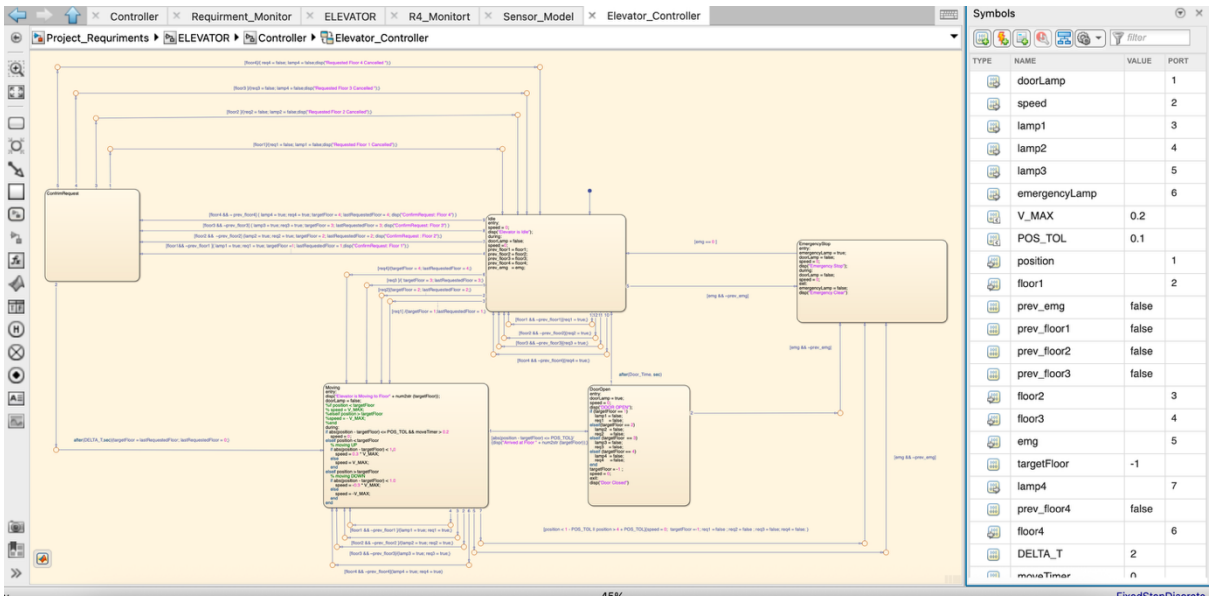


Figure 6:Elevator Controller

3.2 Simulink Model Construction

The Simulink model was built in a structured and modular way to avoid confusion and to keep different functionalities separated. Instead of using a single large model, multiple models were created, each serving a specific purpose such as control logic, physical behaviour, requirement checking, and hardware interaction.

The elevator controller is implemented using a State-flow chart. This chart handles all main behaviours of the system, including floor request confirmation, elevator movement, door opening and closing, and emergency handling. The controller receives button inputs and sensor signals and generates outputs such as speed, door status, and floor indicator lamps.

Button inputs are converted into Boolean signals before entering the controller. This avoids unstable behaviour and makes the model compatible with both simulation and formal verification. The same controller logic is reused in different models, such as the pure Simulink model, Arduino hardware model, and the SLDV model. This ensures consistent behaviour across all testing environments.

A fixed-step discrete solver with a sample time of 0.01 seconds is used. This choice allows correct timing behaviour for features such as door open duration, request confirmation time, and requirement monitoring, while also being compatible with Simulink Design Verifier.

4. Design Decisions and Rationale

This section explains the key design decisions made during development and the reasons behind them.

4.1 Choice of State-Flow

State-Flow was chosen instead of pure Simulink logic because it allows explicit representation of operating modes and transitions. Timing-based logic such as request confirmation and door open duration is easier to implement and debug using State-Flow states and transitions.

4.2 No Request Queue Implementation

A deliberate decision was made **not to implement a request queue**. Although real elevators typically support multiple queued requests, this was not required by the project specification.

Reasons for this choice:

- Reduced controller complexity
- Deterministic behaviour
- Easier formal verification
- Clear mapping between request and response

Only one request is handled at a time.

4.3 Request Confirmation and Cancellation Logic

To handle accidental button presses, a confirmation mechanism was implemented. After a button is pressed, the system enters a confirmation state for a duration DELTA_T.

If the same button is pressed again within DELTA_T = 2 s, the request is cancelled.

The value of 2 seconds was chosen because:

- it is long enough for a user to react,
- it is easy to observe during testing,
- it avoids overly sensitive cancellation behaviour.

4.4 Discrete-Time Design Choice

The entire system was implemented using a **fixed-step discrete solver** with a sample time of **0.01 s**.

This decision was driven by:

- compatibility with Simulink Design Verifier,
- suitability for code generation,
- predictable and consistent timing behaviour.

A sample-time mismatch initially caused incorrect door timing behaviour, which was later fixed by unifying all sample times.

4.5 Separation into Multiple Models

Multiple models were used to:

- avoid incompatibilities between Arduino blocks and SLDV,
- keep verification logic separate from controller logic,
- improve clarity and maintainability.

Each model serves a single, well-defined purpose.

---

5 Motor–Elevator Physical Model

5.1 Modelling Approach

The elevator's physical behaviour was modelled using a **simplified kinematic abstraction**. The controller outputs a speed command, which is integrated to obtain the elevator position.

This approach is sufficient for verifying stopping behaviour, timing constraints, and safety requirements.

5.2 Model Structure

The physical model consists of the following steps:

1. **Speed command (speed)** generated by the controller
2. **Discrete integration** to compute position

3. **Saturation** limiting position to the range [1, 4]
4. **Position tolerance (POS_TOL)** to detect floor alignment robustly

---

5.3 Elevator Physical Model

The physical behaviour of the elevator is modelled using a simple discrete-time approach. The controller outputs a speed signal, which represents the direction and magnitude of elevator movement. This speed is first scaled using a gain block and then passed through a discrete integrator to compute the elevator position over time.

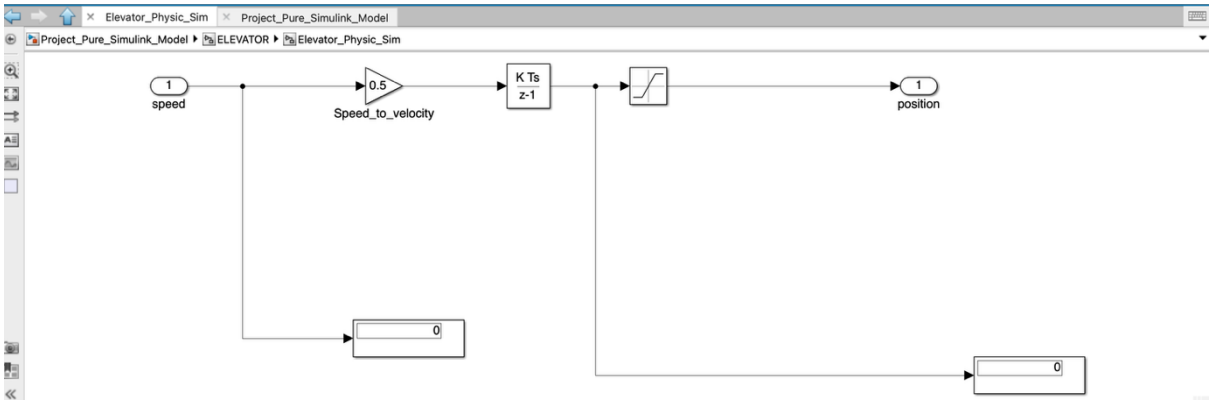


Figure 7 :Elevator_Physic_Sim

A saturation block is used to limit the position signal within valid bounds of the elevator shaft. The final position value is rounded to represent discrete floor levels. This simplified physical model is sufficient to verify the logical behaviour of the controller without introducing unnecessary complexity.

A discrete integrator is used instead of a continuous one to ensure compatibility with SLDV and fixed-step simulation. This model allows clear observation of elevator movement, stopping behaviour, and floor alignment.

5.4 Design Parameters

Parameter	Description	Rationale
V_MAX	Maximum elevator speed	Visible motion without unrealistic dynamics
POS_TOL	Position tolerance	Avoids oscillation at floor boundaries
Sample time	0.01 s	Required for timing accuracy
Saturation	[1, 4]	Matches physical floor limits

5.5 Rationale for Simplification

A detailed mechanical model was not used because:

- the project focuses on controller correctness,
- a simplified model is sufficient for verification,
- it avoids unnecessary instability and complexity.

In simulation setup, this model replaces the ultrasonic distance sensor to obtain the position , while in the hardware setup the position signal is obtained directly from the ultrasonic sensor and mapped to floor positions.

5.6 Ultrasonic Distance-Based Position Feedback

In the hardware implementation, an ultrasonic distance sensor was used to obtain position feedback of the elevator. The sensor measures the distance between the elevator car and a fixed reference point along the shaft. This distance value is read by the Arduino and passed to the Simulink model during execution.

The raw distance signal is scaled and mapped to a position value that represents the current elevator height. This position is then used by the controller to determine the direction of motion and to detect when the elevator has reached the requested floor. To make floor detection more robust, the position signal is rounded only for floor indication and stopping logic, while the continuous value is kept for movement control.

This approach made it possible to reuse the existing State-flow controller without major changes, while still achieving reliable stopping behaviour in hardware. Minor noise in the sensor signal was observed, but it did not affect the overall correctness of the system.

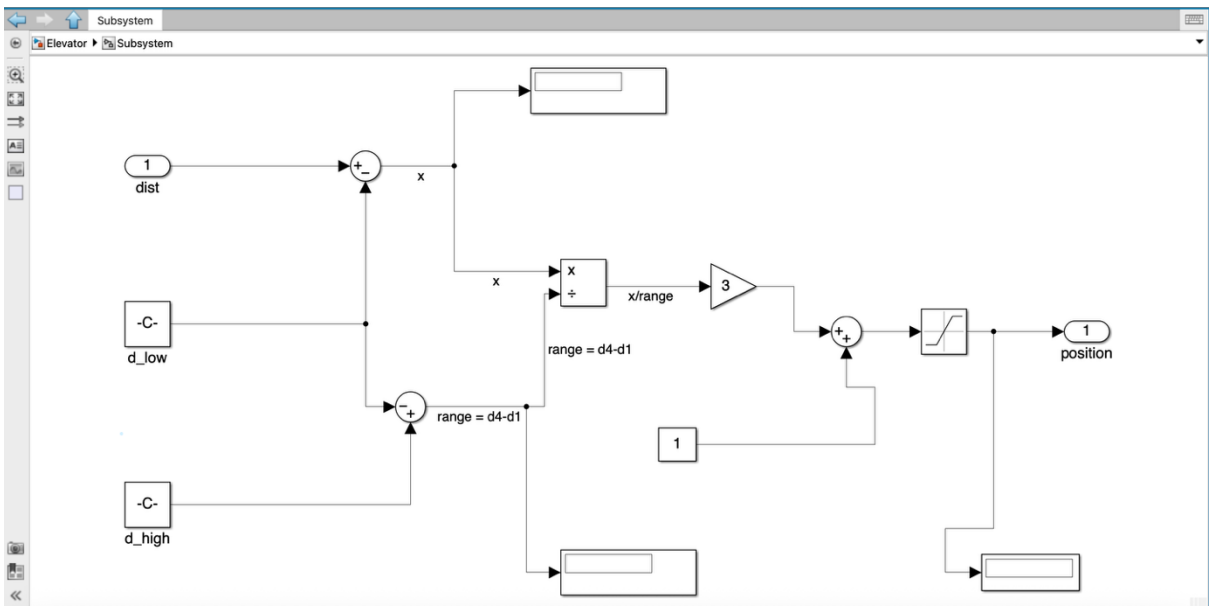


Figure 8:Ultraonic_Position_Physics

During calibration, reference distance values were recorded at known floor positions. The distance measured at floor 1 was taken as d_low= 0.0314, and the distance measured at floor 4 was taken as d_high = 0.7000. These values were used as fixed reference points to map the measured ultrasonic distance to the corresponding elevator position. Using these reference distances helped ensure consistent and repeatable stopping behaviour across all floors.

---

6 Formalization of Requirements (R1–R4)

To verify the correctness of the elevator controller, the system requirements were formalized as **separate State-Flow requirement monitors**. Each requirement was implemented as an independent monitor that observes relevant signals from the controller and the physical elevator model.

Each monitor produces a Boolean signal (R*_ok) that indicates whether the requirement is currently satisfied. If a violation is detected, the corresponding R*_ok signal switches to false.

The requirements were modelled independently from the controller logic in order to keep the design modular and to allow automatic verification using Simulink Design Verifier (SLDV).

6.1 Requirement R1 – Door Safety During Motion

Requirement Description (R1)

R1:
The elevator door must not be open while the elevator is moving between floors.

This requirement is a basic safety condition: passengers must never be exposed to an open door unless the elevator is stationary at a valid floor.

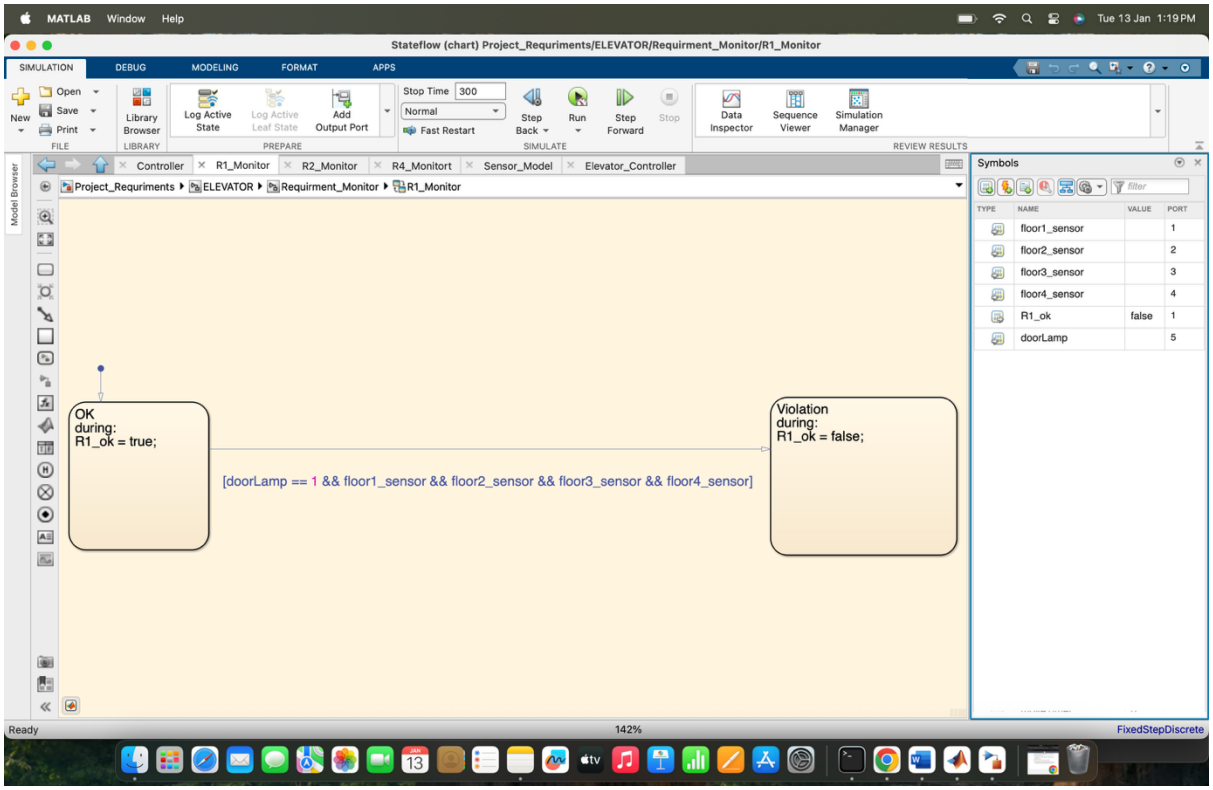


Figure 9:R1

Signals Used

The R1 monitor observes the following signals:

- doorLamp – indicates whether the door is open
- floor1_sensor, floor2_sensor, floor3_sensor, floor4_sensor – indicate whether the elevator is aligned with a floor

---

Formalization in State-Flow

- The monitor starts in an **OK** state with R1_ok = true
- A transition to the **Violation** state occurs if:
  - the door is open (doorLamp == 1)
  - while none of the floor sensors are active

This condition corresponds to the door being open while the elevator is not at any floor, which implies motion or misalignment.

Violation Meaning

A violation of R1 means:

- the door was opened while the elevator was moving or between floors

Once the violation state is reached, R1_ok is set to false and remains false for the rest of the simulation.

Results

After fixing timing and sample-time inconsistencies, no R1 violations were observed. The controller correctly ensures that the elevator stops before opening the door.

6.2 Requirement R2 – Door Open Time

Requirement Description (R2)

R2:
When the elevator door opens, it must remain open for at least a specified minimum time.

In this project, the required door open time was defined as **5 seconds**.

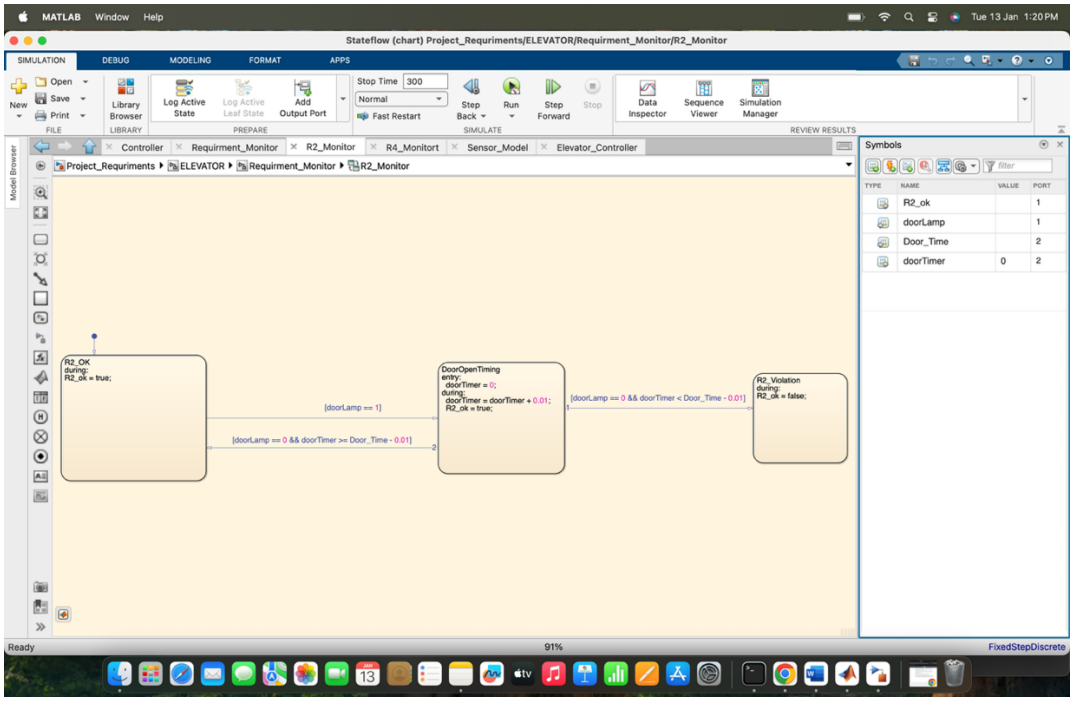


Figure 10:R2

Signals Used

The R2 monitor observes:

- doorLamp – door open indicator
- Door_Time – configured door open duration
- an internal timer (doorTimer)

Formalization in State-Flow

The R2 monitor consists of three states:

1. **R2_OK**
   - Default state where the requirement is satisfied
2. **DoorOpenTiming**
   - Entered when doorLamp == 1
   - A timer is incremented at each simulation step
   - R2_ok remains true during this period
3. **R2_Violation**
   - Entered if the door closes (doorLamp == 0)
   - before the timer reaches the required door open time

Sample Time Consideration

Initially, an incorrect sample time caused the door to remain open for only **0.5 seconds instead of 5 seconds**.
This was due to the monitor running at 0.1 s while the system ran at 0.01 s.

After synchronizing the sample times, the door timing behaved correctly.

Results

After correcting the sample time mismatch, no further R2 violations were detected.

6.3 Requirement R3 – No Stop at Unrequested Floors

Requirement Description (R3)

R3:
The elevator must not stop at a floor that has not been requested.

This requirement ensures correct handling of user requests and prevents unnecessary or unexpected stops.

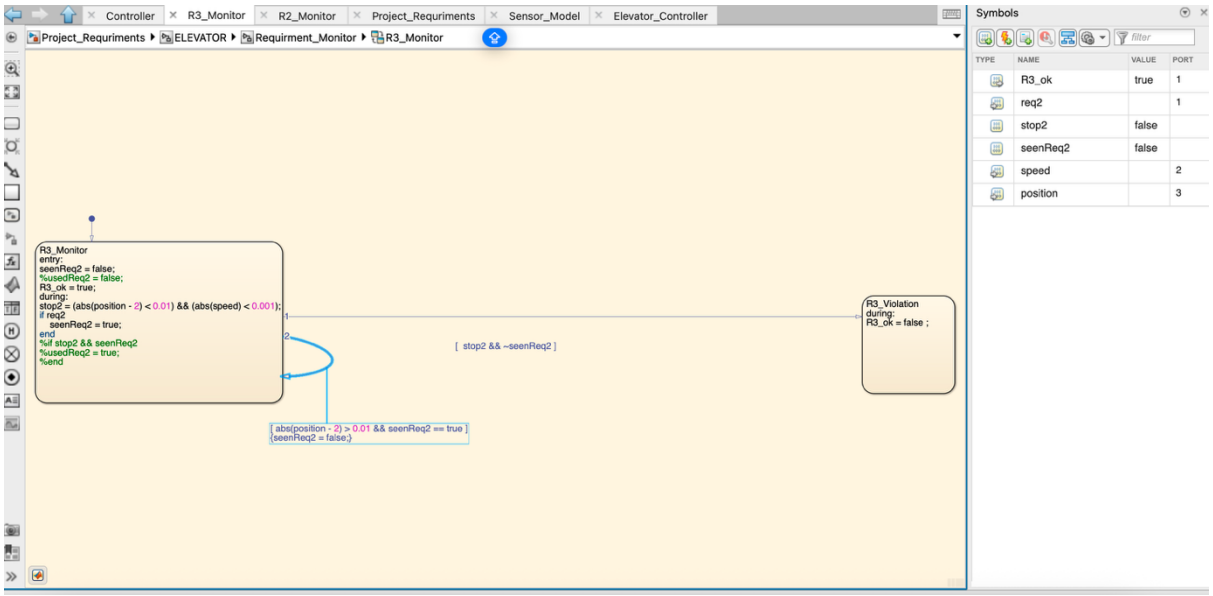


Figure 11:R3

Signals Used

The R3 monitor observes:

- position
- speed
- request signals (req1, req2, req3, req4)
- internal flags (seenReq2, etc.)

Formalization in State-Flow

The monitor tracks whether a stop occurs at a specific floor:

- A stop is detected when:
  - the elevator position is close to a floor (within tolerance)
  - and the speed is approximately zero

If such a stop occurs **without a corresponding request**, the requirement is violated.

To avoid false detections:

- the monitor resets its internal flags once the elevator leaves the floor area

---

Violation Meaning

A violation of R3 means:

- the elevator stopped at a floor that the user never requested

---

Results

No unintended stops were observed in the corrected controller. All stops occurred only at requested floors.

6.4 Requirement R4 – Maximum Travel Time / Step Count

Requirement Description (R4)

R4:
After a request is made, the elevator must reach the requested floor within a maximum allowed number of steps (or time ticks).

This requirement ensures that the elevator does not take unreasonably long to reach a destination.

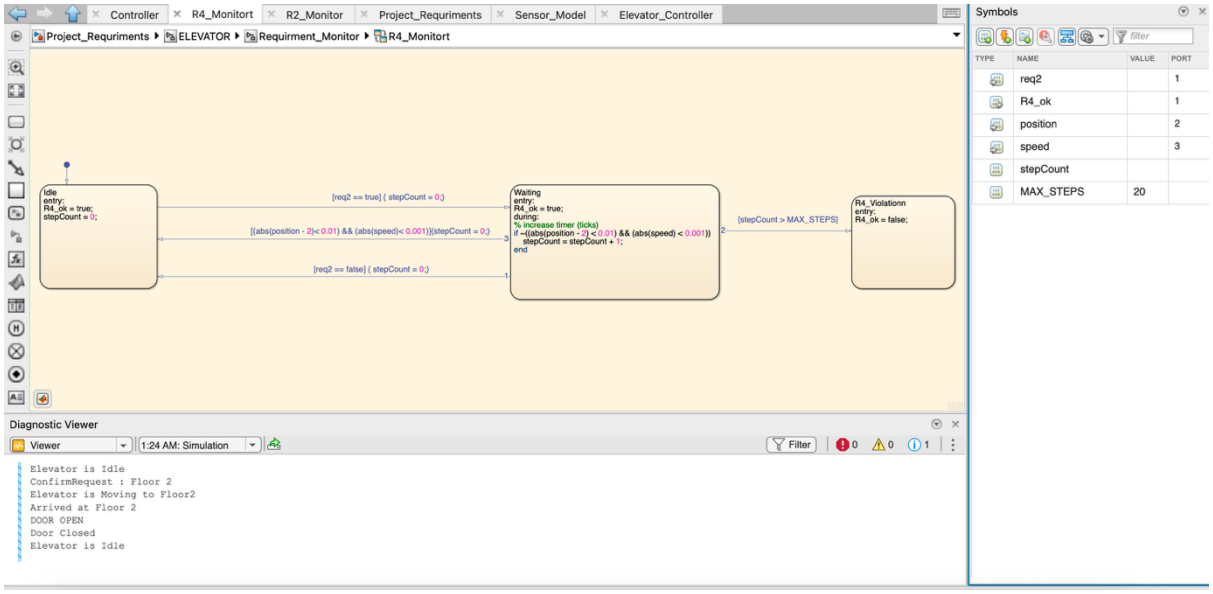


Figure 12:R4

Signals Used

The R4 monitor observes:

- req2 (or other request signals)
- position
- speed
- an internal counter (stepCount)
- a configured maximum (MAX_STEPS)

Formalization in State-Flow

The R4 monitor operates as follows:

1. **Idle**
   - Waits for a request
   - Initializes stepCount = 0
2. **Waiting**
   - Increments stepCount at each simulation tick
   - Continues until the elevator reaches the requested floor
3. **Violation**
   - Entered if stepCount > MAX_STEPS

Violation Meaning

A violation of R4 means:

- the elevator took too long to reach the requested floor

This can indicate issues with speed limits, motion logic, or controller delays.

Results

During early testing, R4 violations helped reveal unrealistic timing assumptions. After tuning speed (V_MAX) and timing parameters, the elevator consistently reached its target within the allowed step count.

6.5 Summary of Requirement Verification

Requirement	Description	Result
R1	Door not open while moving	Passed
R2	Door remains open for required time	Passed (after fix)
R3	No stop at unrequested floors	Passed
R4	Reach target within maximum time/steps	Passed

6.6 Discrepancies and Fixes

The main discrepancy detected was incorrect door open duration (R2). The door remained open for 0.5 seconds instead of 5 seconds.

Cause:
Mismatch between controller and monitor sample times.

Resolution:
Unified sample time to 0.01 s and re-verified behaviour.

After this fix, no further violations were observed.

---

7 Simulink Design Verifier (SLDV)

Purpose of Using SLDV

Simulink Design Verifier (SLDV) was used to **automatically generate test cases** and to **analyse requirement coverage**.

The motivation for using SLDV in this project was:

- To explore corner cases that are difficult to trigger manually
- To validate requirement monitors under automatically generated input scenarios
- To assess how well the formal requirements are exercised by the system behaviour

---

7.1 SLDV-Compatible Model Design

A dedicated SLDV model was created with the following constraints:

- Only **Inport** and **Outport** blocks were used
- No Dashboard blocks, Displays, or Arduino I/O were included
- The controller, physical model, and requirement monitors were connected in a single verification model

This separation was necessary because SLDV does not support interactive or hardware-specific blocks.

7.2 Test Generation and Coverage Results

SLDV was configured to generate tests for the formal requirements (R1–R4). Due to the complexity of the State-Flow logic and the number of states, the analysis reached the **maximum processing time limit**.

As a result:

- Partial coverage was achieved
- Several test cases were successfully generated
- Some conditions were reported as unreachable or undecidable within the time limit.
- Due to the time-bounded nature of the analysis, an exact numerical coverage percentage was not reported. The achieved partial coverage was considered sufficient to validate the main operational and safety requirements of the system.

Despite this limitation, the generated tests were sufficient to:

- Trigger normal operation scenarios
- Validate door timing behaviour
- Detect timing-related inconsistencies during early development

---

7.3 Limitations Observed

The following limitations were observed during SLDV analysis:

- State-space explosion due to multiple interacting State-Flow charts
- Timeouts for deep exploration of timing-based requirements
- Conservative reachability analysis for continuous/discrete hybrid behaviour

These limitations are expected for systems with timing constraints and event-driven logic.

7.4 Resolution of Detected Issues

One discrepancy detected during verification was a **sample-time mismatch** between the controller and the physical model. This resulted in incorrect door open duration.

This issue was resolved by:

- aligning the controller and model sample time to 0.01 seconds.
- Re-running the verification after correction

After this fix, no critical requirement violations were detected.

---

8 Results & Conclusion

Logging and Simulation Comparison

Two simulation setups were used for logging and validation. One setup included the motor command generator and elevator physical model, while another setup excluded the motor logic to focus on controller behaviour.

In the pure Simulink model without motor involvement, correct behaviour such as request confirmation, cancellation, elevator movement, and door operation was observed clearly through logged signals and diagnostic messages.

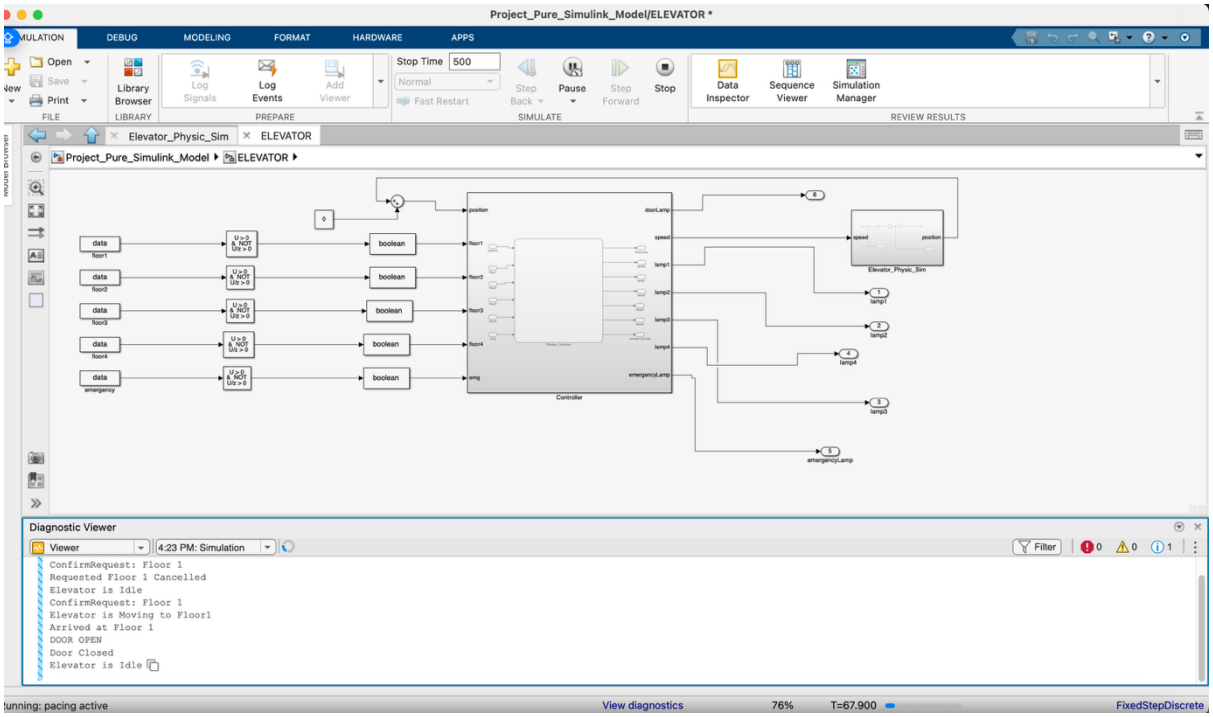


Figure 13:Without_Motor_Involvment

When the motor command generator was included, the system focused on validating signal flow between the controller and motor interface (IN1, IN2, ENA). In this setup, some high-level behaviours such as request cancellation were not visible due to abstraction at the motor level. This difference is acceptable and is documented in the report.

Both simulations confirm that the controller behaves as expected under different configurations.

This project demonstrates a structured approach to elevator controller design using Simulink and State-Flow. By separating control logic, physical modelling, hardware execution, and verification, the system remains understandable and verifiable.

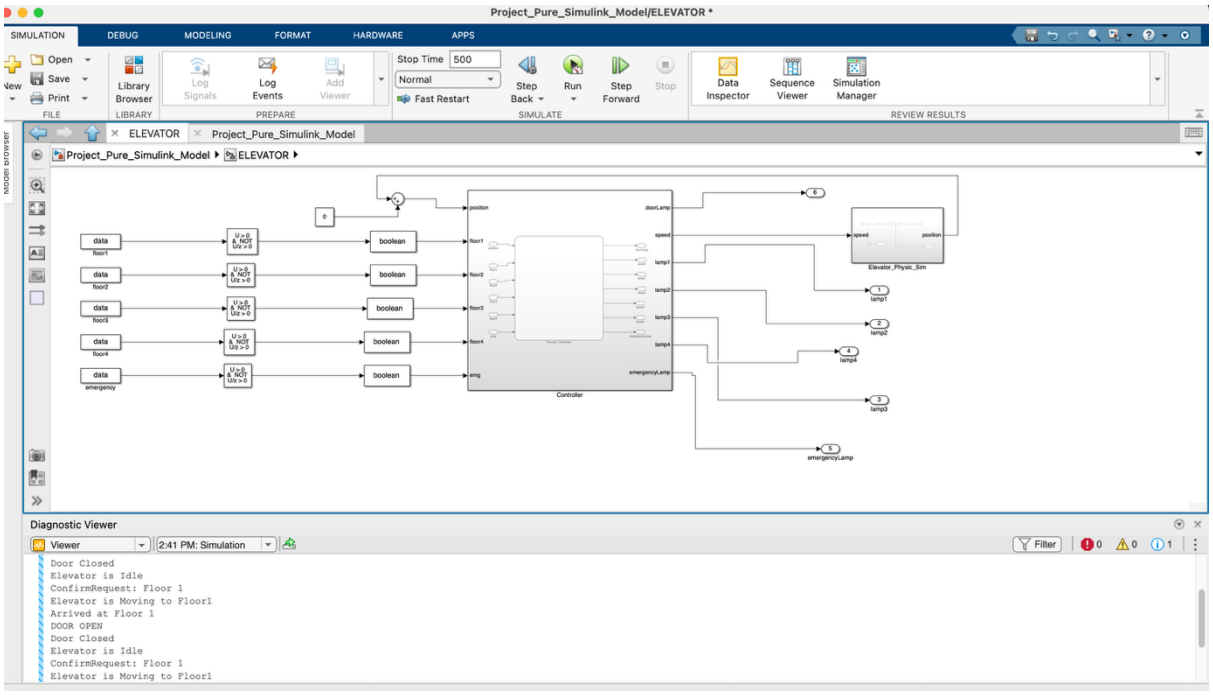


Figure 14:With_Motor_Involvment

Formal requirement monitoring and automated test generation were used to validate correctness. Simplifications were made deliberately to focus on controller behaviour rather than mechanical realism.

The final implementation demonstrates that the elevator controller works reliably in both simulation and hardware execution. The integration of ultrasonic distance feedback allows the elevator to stop at the requested floor without relying on idealized position signals. Although the speed profile is not perfectly smooth due to the simplified control approach and sensor noise, the overall behaviour satisfies the project requirements. Further improvements could include smoother speed control, but these were considered outside the scope of this project.

Overall, the implemented system satisfies the project requirements and shows consistent behaviour in both simulation and hardware execution.

**Note on Floor Indexing

The elevator starts at floor 1 by default. During simulation, signals may display activity for floor 1 even when higher floors are selected. This is due to initial conditions and signal naming conventions and does not affect system correctness. Floor requests and arrivals are verified using position and sensor logic rather than display labels.